What it is: A function is a reusable block of code.

You write it once => use it many times.

In [5]:
```
#basic function
def say_hello():              # def => creating function
    print("hello sheela")     #say_hello → function name
                                #() → inputs (empty here)
say_hello()
```

```
hello sheela
```

In [6]:
```
#2. Function with Input (parameter)
def greet(name):
    print("hello",name)    #Same function works for different people
greet("sheela")
greet("seema")
```

```
hello sheela
hello seema
```

In [9]:
```
def info(name):
    print("i am ", name)
greet("sheela")
```

```
hello sheela
```

In [11]:
```
#Function with Return #return gives value back,You can store it
def add(a,b):
    return a+b
print(add(33,44))
```

```
77
```

In [3]:
```
def add(a,b):
    return a+b

print(add(32,3))
```

```
35
```

In [25]:
```
#real life eg
def login(username, password):
    if username == "sheela" and password == "9876":
        return "login success"
    else:
        return "login failed, try again"

user = input("Enter username: ")
pwd = input("Enter password: ")

print(login(user, pwd))
```

```
Enter username: sheela
Enter password: eeee
login failed, try again
```

print() _____ return Shows output Gives value back Cannot reuse Can reuse Not stored Stored in variable

What is an Iterator?

An iterator is something that lets you go through data one item at a time

In [33]:
```
nums=[10,20,30,40]

for n in nums:
    print(n)
```

```
10
20
30
40
```

In [ ]:
```
nums=[10,20,30,40]
```

In [35]:
```
nums = [10, 20, 30, 40] #iter() → creates iterator, next() → gives next value

it = iter(nums)

print(next(it))
print(next(it))
print(next(it))
print(next(it)) #it  ends when no more value is left
```

```
10
20
30
40
```

Iterable → something you can loop (list, tuple) Iterator → object that gives values one by one iter() → create iterator next() → get next value

What is a Generator?

A generator is a function that gives values one by one using yield

It is a special type of iterator

In [38]:
```
def numbers():
    yield 1
    yield 2
    yield 3

for n in numbers():
    print(n)
```

```
1
2
3
```

What is yield? yield works like return BUT…

return → ends function yield → pauses function and continues later

File Operations (Read & Write Files) What it is => Used to store and read data from files Like working with .txt, .csv, etc.

In [ ]:
```
#Read a file

with open("data.txt", "r") as f:
    content = f.read()
    print(content)
```

```
Introduction to Pandas
What is Pandas?

Library used to handle data like Excel tables

Very important for:
Data analysis
CSV handling

Pandas helps you:

read it
filter it
analyze it
modify it
```

```
Main Uses of Pandas
 Read data (CSV, Excel)
```

In [46]:
```
import pandas as pd

df = pd.read_csv("ds_salaries.csv")
```

In [49]:
```
df.head
```

Out[49]:
```
<bound method NDFrame.head of      Unnamed: 0  work_year experience_level emplo
yment_type  \
0             0       2020               MI              FT
1             1       2020               SE              FT
2             2       2020               SE              FT
3             3       2020               MI              FT
4             4       2020               SE              FT
..          ...        ...              ...             ...
602         602       2022               SE              FT
603         603       2022               SE              FT
604         604       2022               SE              FT
605         605       2022               SE              FT
606         606       2022               MI              FT

                     job_title  salary salary_currency  salary_in_usd  \
0                Data Scientist   70000             EUR          79833
1    Machine Learning Scientist  260000             USD         260000
2             Big Data Engineer   85000             GBP         109024
3          Product Data Analyst   20000             USD          20000
4     Machine Learning Engineer  150000             USD         150000
..                          ...     ...             ...            ...
602               Data Engineer  154000             USD         154000
603               Data Engineer  126000             USD         126000
604                Data Analyst  129000             USD         129000
605                Data Analyst  150000             USD         150000
606                AI Scientist  200000             USD         200000

    employee_residence  remote_ratio company_location company_size
0                   DE             0               DE            L
1                   JP             0               JP            S
2                   GB            50               GB            M
3                   HN             0               HN            S
4                   US            50               US            L
..                 ...           ...              ...          ...
602                 US           100               US            M
603                 US           100               US            M
604                 US             0               US            M
605                 US           100               US            M
606                 IN           100               US            L

[607 rows x 12 columns]>
```

In [48]:
```
df.tail()
```

Out[48]:

| | Unnamed: 0 | work_year | experience_level | employment_type | job_title | salary | salary_currency |
|---|---|---|---|---|---|---|---|
| **602** | 602 | 2022 | SE | FT | Data Engineer | 154000 | USD |
| **603** | 603 | 2022 | SE | FT | Data Engineer | 126000 | USD |
| **604** | 604 | 2022 | SE | FT | Data Analyst | 129000 | USD |
| **605** | 605 | 2022 | SE | FT | Data Analyst | 150000 | USD |
| **606** | 606 | 2022 | MI | FT | AI Scientist | 200000 | USD |

Module

=> A single .py file with functions/variables

Why use?

Avoid repeating code Keep code organized

Library

=> A collection of modules built for a purpose

Examples:

pandas → data handling numpy → math matplotlib → graphs

Module → 1 file Package → folder of files Library → big ready-made tool

2. Errors & Exception Handling When code breaks → we handle it instead of crashing

=> Types of errors

In [ ]:
```
#Syntax Error (writing mistake)

if True
    print("Hi") # missing : , code wont run
```

In [ ]:
```
# Runtime Error (happens while running)

print(10 / 0)

# =>Crash → ZeroDivisionErro
```

In [ ]:
```
# Logical Error (no crash, wrong output)

print(2 + 2) #expected 5 #Code runs, but result is wrong
```

What is Exception Handling?

=> It means handling errors so program does NOT crash

In [ ]:
```
# Basic Structure
try:
    # risky code
except:
    # runs if error happens
```

In [52]:
```
#If user enters:0 → division error,abc → invalid input

👉 Program does NOT crash
try:
    num=int(input("Enter number"))
    print(10/num)
except:
    print("something went wrong")
```

```
Enter number0
something went wrong
```

In [54]:
```
try:
    num = int(input("Enter number: "))
    print(10 / num)

except ZeroDivisionError:
    print("Cannot divide by zero")

except ValueError:
    print("Please enter a number")
```

```
Enter number: 3
3.3333333333333335
```

Database Connection Connect Python to database (like SQL)

A database connection is used to connect your Python program to a database so you can:

store data retrieve data update/delete data

In [ ]:
```
Basic Steps
Connect to database
Create cursor
Execute query
Fetch data
Close connection

----------------------------
commit() → saves changes
cursor() → runs SQL queries
fetchall() → gets data
```

In [2]:
```
import sqlite3
```

In [3]:
```
#connect to database
db = sqlite3.connect("my_database4.db")
cursor = db.cursor()
```

In [4]:
```
# drop table if exists
cursor.execute("DROP TABLE IF EXISTS grades2")
```

Out[4]:
```
<sqlite3.Cursor at 0x1c95f610840>
```

In [5]:
```
# create table
cursor.execute("CREATE TABLE grades2(id INTEGER, name TEXT, score INTEGER)")

# insert data (NEW NAMES)
cursor.execute("INSERT INTO grades2 VALUES (201, 'Aarav', 91)")
cursor.execute("INSERT INTO grades2 VALUES (202, 'Priya', 88)")
cursor.execute("INSERT INTO grades2 VALUES (203, 'Kabir', 76)")
cursor.execute("INSERT INTO grades2 VALUES (204, 'Ananya', 94)")
cursor.execute("INSERT INTO grades2 VALUES (205, 'Rohan', 82)")

# save changes
db.commit()
```

In [6]:
```
# SELECT ALL
print("All Records:")
results = cursor.execute("SELECT * FROM grades2 ORDER BY id")
for row in results:
    print(row)
print("-" * 50)
```

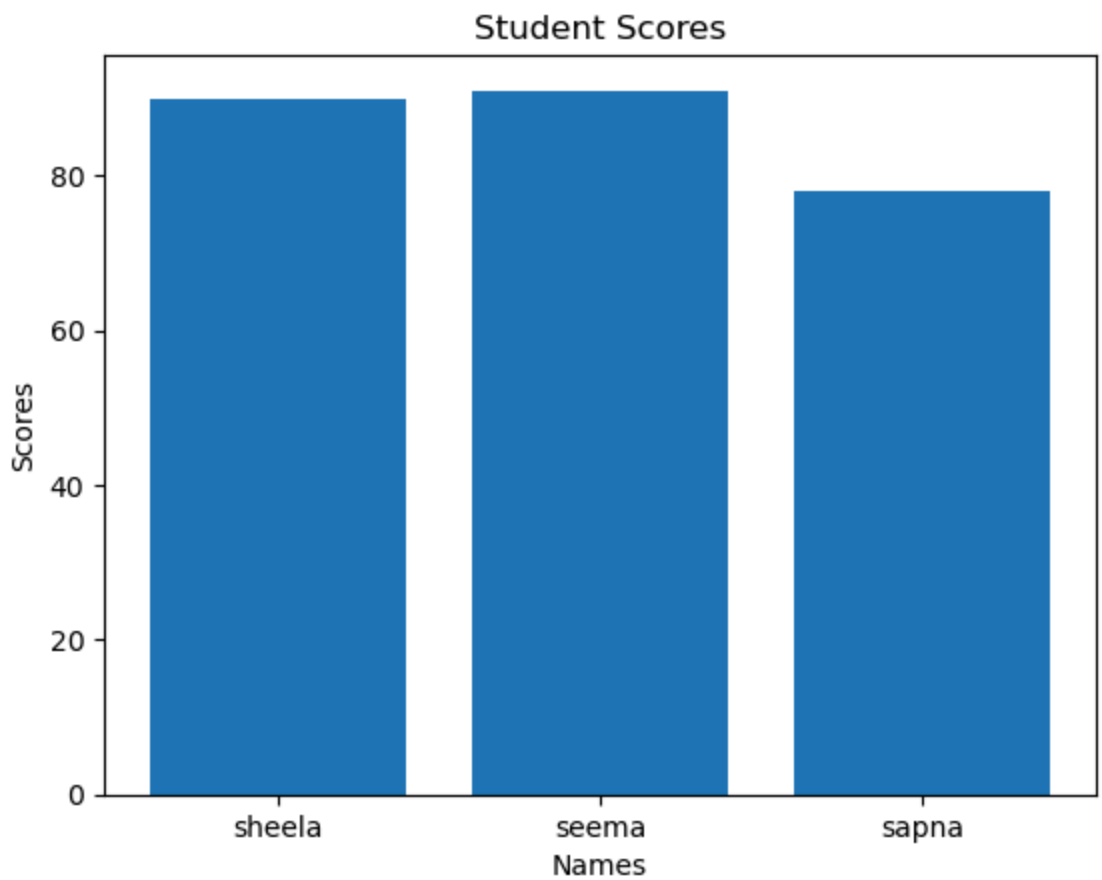
```
All Records:
(201, 'Aarav', 91)
(202, 'Priya', 88)
(203, 'Kabir', 76)
(204, 'Ananya', 94)
(205, 'Rohan', 82)
--------------------------------------------------
```

```
Data Visualization
What it is

Data visualization means showing data using charts, graphs, or visuals instead
of raw numbers.

=>Helps you:

understand data quickly
find patterns
make decisions
```

```
Why use Data Visualization?
Easy to understand
Shows trends and patterns
Helps compare data
Useful for reports and presentations
```

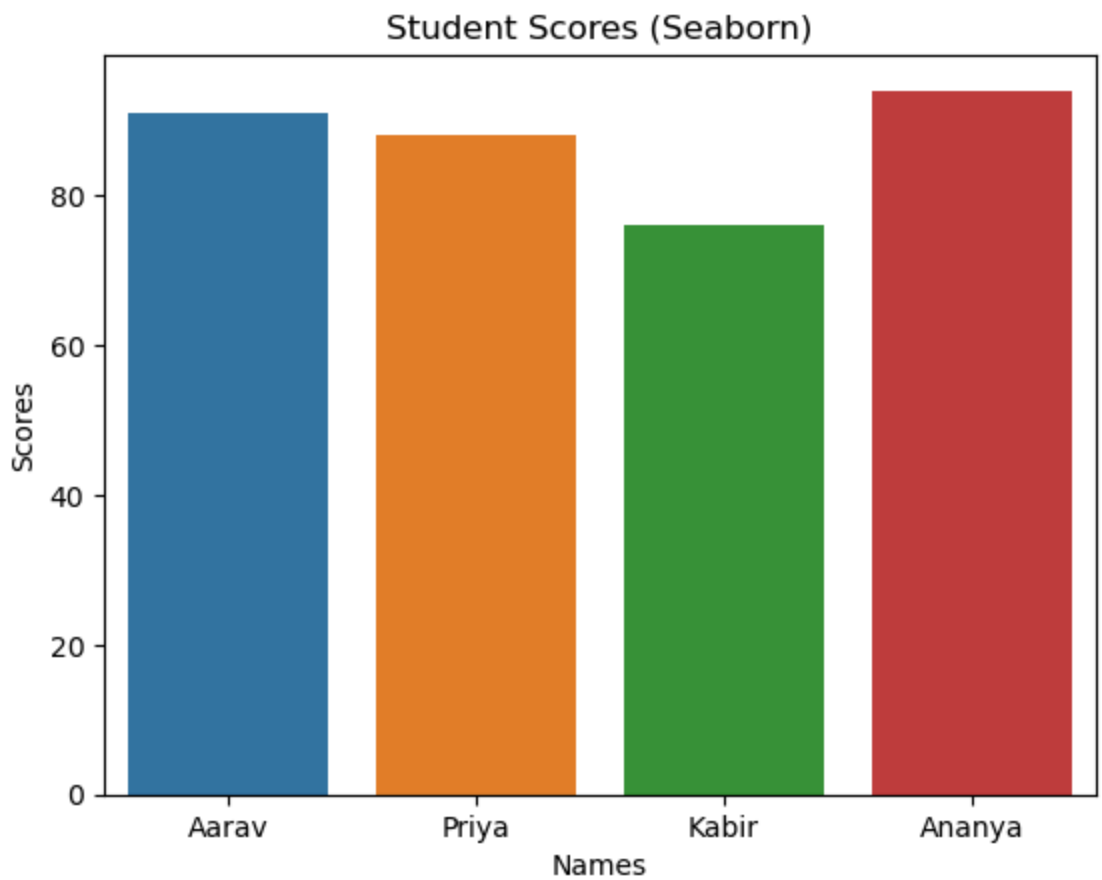
```
Tools in Python
1. Matplotlib
Basic plotting library
Used for simple graphs
2. Seaborn
Built on Matplotlib
More attractive graphs
3. Pandas
Used to handle data
Can create simple plots
```

In [12]:
```
import matplotlib.pyplot as plt

names = ["sheela", "seema", "sapna"]
scores = [90, 91, 78]
plt.bar(names, scores)
plt.title("Student Scores")
plt.xlabel("Names")
plt.ylabel("Scores")

plt.show()
```




```
Key Terms
X-axis → horizontal line
Y-axis → vertical line
Label → description of axis
Legend → explains colors
```

In [14]:
```
import seaborn as sns
import matplotlib.pyplot as plt

names = ["Aarav", "Priya", "Kabir", "Ananya"]
scores = [91, 88, 76, 94]
sns.barplot(x=names, y=scores)
plt.title("Student Scores (Seaborn)")
plt.xlabel("Names")
plt.ylabel("Scores")
plt.show()
```




In [ ]:

In [ ]:

In [ ]:

In [ ]: